A PROJECT REPORT

ON

CAMPUS LOST & FOUND PORTAL

Submitted in partial fulfillment of the requirements
for the award of the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING

Submitted by

CHANDAN SINGH
UID: 24BCS12108

VIVEK MANHAS
UID: 24BCS11481

Under the Guidance of

Er. Venkatesh K
Assistant Professor



Chandigarh University
2024-2028

BONAFIDE CERTIFICATE

This is to certify that the project report entitled **"Campus Lost & Found Portal"** submitted by **Chandan Singh (UID: 24BCS12108)** and **Vivek Manhas (UID: 24BCS11481)** to the Chandigarh University, in partial fulfillment of the requirement for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** is a bonafide record of the work carried out by them under my supervision and guidance. The contents of this report, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

SIGNATURE **SIGNATURE**

HEAD OF THE DEPARTMENT
DEPARTMENT OF COMPUTER
SCIENCE & ENGINEERING
CHANDIGARH UNIVERSITY

SUPERVISOR

Submitted for the project viva-voce examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide Er. Hartej Singh for the continuous support, guidance and valuable suggestions throughout the duration of this project. Without his/her help it would have been difficult for us to complete this work properly.

We are also thankful to the Head of the Department and all the faculty members of Computer Science and Engineering department for providing the necessary facilities and encouragement during the project work. Their support motivated us to complete the project in a systematic manner.

We would also like to thank our parents and friends who supported us during the project work. Their motivation and patience helped us a lot in completing this project on time.

Finally, we are grateful to everyone who helped us directly or indirectly in the successful completion of this project. We have tried our best to complete the work sincerely and honestly.

Chandan Singh
UID: 24BCS12108

Vivek Manhas
UID: 24BCS11481

TABLE OF CONTENTS

ACKNOWLEDGEMENT ii
TABLE OF CONTENTS iii
LIST OF FIGURES iv
LIST OF TABLES v
ABSTRACT vi
ABBREVIATIONS vii
CHAPTER 1 INTRODUCTION 1
1.1 Identification of Client and Need 1
1.2 Relevant Contemporary Issues 2
1.3 Problem Identification 2
1.4 Task Identification 3
1.5 Timeline 3
1.6 Organization of the Report 4
CHAPTER 2 LITERATURE SURVEY 5
2.1 Introduction to Lost and Found Systems and Web Applications 5
2.2 Importance of Authentication, Privacy and Real-Time Communication 5
2.3 Technologies Available 6
2.4 Related Work 6
2.5 Summary and Problem Definition 7
2.6 Goals and Objectives 7
CHAPTER 3 DESIGN FLOW / PROCESS 9
3.1 Concept Generation 9
3.2 Selection of Technology Stack 9
3.3 Design Constraints 10
3.4 Alternative Designs 10
3.5 Best Design Selection 11
3.6 Implementation Plan 11
CHAPTER 4 RESULTS ANALYSIS AND VALIDATION 13
4.1 Tools Used 13
4.2 System Architecture and Project Structure 13
4.3 Modules Implemented 14
4.4 API Endpoints 15
4.5 Positive Test Cases 16
4.6 Negative and Security Test Cases 17
4.7 Environment Variables and Deployment 18
4.8 Issues Found and Fixed 19
4.9 Detailed Observations 20
CHAPTER 5 CONCLUSION AND FUTURE WORK 22
5.1 Conclusion 22

5.2 Deviation from Expected Results 22
5.3 Future Work 22
REFERENCES 24
APPENDIX 25
USER MANUAL 27

LIST OF FIGURES

Figure No.	Figure Caption
Figure 3.1	Overall Architecture of Campus Lost & Found Portal
Figure 4.1	Project Folder Structure (client/ and server/)
Figure 4.2	Login Page
Figure 4.3	Register Page
Figure 4.4	Dashboard
Figure 4.5	Lost Items Listing
Figure 4.6	Found Items Listing
Figure 4.7	My Listings Page
Figure 4.8	Anonymous Chat Interface
Figure 4.9	Admin Overview Page
Figure 4.10	Deployment Architecture (Render + Vercel)

LIST OF TABLES

Table No.	Table Caption
Table 3.1	Comparison of Available Technology Stacks
Table 4.1	List of API Endpoints
Table 4.2	List of Positive Test Cases
Table 4.3	List of Negative and Security Test Cases
Table 4.4	Issues Found and Fixed During Testing

ABSTRACT

Lost and found systems are an important part of campus life. Students frequently lose items such as identity cards, books, water bottles, chargers and other belongings. Traditional notice boards and social media groups are inefficient, lack privacy and do not provide a reliable way for the finder and the owner to communicate safely.

In this project, a full-stack Campus Lost & Found Portal has been developed. Students can register, log in, create lost or found listings, view all listings, manage their own listings and chat privately with the other party through anonymous chat rooms. An admin role is also provided for overview statistics.

The frontend is built with React and Vite. The backend uses Node.js with Express for the REST API and Socket.IO for real-time messaging. JWT is used for authentication and bcrypt for password hashing. Role-based access control protects both frontend routes and backend endpoints. For the current demo the data is stored in memory; Mongoose models have been prepared so that the system can later be migrated to MongoDB without major changes.

The application has been deployed with the backend on Render and the frontend on Vercel. Source code is available on GitHub. The project gave us practical experience of building a complete full-stack application with authentication, real-time communication and proper security practices.

ABBREVIATIONS

S. No.	Abbreviation	Full Form
1	API	Application Programming Interface
2	REST	Representational State Transfer
3	JWT	JSON Web Token
4	CRUD	Create, Read, Update, Delete
5	JSON	JavaScript Object Notation
6	HTTP	Hypertext Transfer Protocol
7	UI	User Interface
8	CORS	Cross-Origin Resource Sharing
9	SPA	Single Page Application
10	WebSocket	WebSocket Protocol
11	bcrypt	Password Hashing Library

CHAPTER 1

INTRODUCTION

1.1 Identification of Client and Need

In a large university campus many students lose personal belongings every day. Items such as ID cards, notebooks, earphones, water bottles, umbrellas and laptop chargers are frequently misplaced in classrooms, libraries, canteens and hostels. At present, students mostly rely on WhatsApp groups, Instagram stories or physical notice boards to report lost or found items.

These methods have several problems. Messages get buried under new posts, personal phone numbers are often shared publicly, and there is no organised way to match a lost item with a found item. Privacy is also a concern because students do not always want to reveal their identity or contact details to strangers.

The need of this project comes from the requirement of a simple, secure and campus-specific platform where students can report lost and found items and contact each other through private, anonymous chat. The system should be easy to use on mobile as well as desktop and should protect the identity of both parties until they decide to share personal information themselves.

This project is developed for the students of Chandigarh University. It tries to solve a real day-to-day problem that almost every student faces at least once during their college life.

1.2 Relevant Contemporary Issues

Modern campuses are large and students move between many locations every day. The volume of lost items is high, yet the existing recovery mechanisms remain informal and unreliable. Some of the contemporary issues related to this problem are:

- Lack of a centralised and trusted platform for lost and found items on campus.
- Privacy risks when students post personal contact numbers on public groups.
- Difficulty in matching a lost item with the correct found item because of unstructured posts.
- No reliable way for the finder and the owner to communicate safely and anonymously.

- Absence of an administrative overview that can help campus authorities understand the scale of the problem.
- Students often give up searching after a few days because the process is time-consuming and frustrating.

These issues show that a dedicated web application with proper authentication, role-based access and real-time private chat can significantly improve the recovery rate of lost items while protecting student privacy.

1.3 Problem Identification

The main problem identified is the absence of a structured, privacy-preserving digital system for lost and found items inside the university campus. Existing solutions (social media groups and notice boards) are temporary, public and do not support safe communication between the two parties.

Another problem is that most student projects that attempt to solve this issue either remain limited to simple listing pages without any chat facility, or they force users to share mobile numbers. Real-time anonymous chat is rarely implemented because it requires additional knowledge of WebSockets and careful security design.

This project addresses these problems by building a complete full-stack portal that includes registration, login, listings, personal management of items, anonymous chat rooms and an admin dashboard.

1.4 Task Identification

The tasks identified for this project are as follows:

1. Decide the overall architecture and technology stack for a full-stack web application.
2. Design and implement user registration and login with JWT authentication and bcrypt password hashing.
3. Create frontend pages for Login, Register, Dashboard, Lost, Found, My Listings, Chat and Admin.
4. Develop REST API endpoints for users, items (lost/found), and chat rooms.

5. Implement Socket.IO based real-time chat with membership checks and anonymous sender labels.

6. Add role-based access control so that only admins can view the statistics page.

7. Prepare Mongoose models (User, Item, ChatRoom, Message) for future MongoDB migration while using an in-memory store for the current demo.

8. Deploy the backend on Render and the frontend on Vercel and configure environment variables.

9. Test the complete system, fix issues related to authentication, routing and chat security, and record the results.

10. Prepare this project report according to the college format and prepare for the viva-voce examination.

1.5 Timeline

The project was completed in the following approximate timeline:

- Week 1 – Requirement analysis, selection of technology stack, creation of project structure (client and server folders) and basic Express + React setup.
- Week 2 – Implementation of authentication (register, login, JWT middleware, protected routes) and basic item listing APIs.
- Week 3 – Frontend pages for Dashboard, Lost, Found and My Listings; Axios integration and environment variable handling.
- Week 4 – Real-time chat with Socket.IO, anonymous labels, membership checks, admin page and deployment on Render and Vercel.
- Week 5 – Testing, fixing of authentication and routing bugs, taking screenshots, writing the project report and preparing for viva.

This timeline helped us complete the work in a systematic manner without last-minute rush. Each week had clear goals which made the progress measurable.

1.6 Organization of the Report

This report is organized into five chapters as per the guidelines given by the department.

Chapter 1 gives introduction, identification of need, contemporary issues, problem identification, tasks and timeline of the project.

Chapter 2 contains literature survey. It explains existing lost and found systems, the importance of authentication and real-time communication, available technologies, related work, problem definition and objectives.

Chapter 3 explains the design process. It covers concept generation, selection of technology stack, design constraints, alternative designs considered, selection of the best approach and the step-by-step implementation plan.

Chapter 4 presents the actual work done. It includes tools used, system architecture, modules implemented, API endpoints, positive and negative test cases, environment variables, deployment details, issues found and fixed, and observations made during the project.

Chapter 5 gives conclusion of the work, any deviation from expected results and possible future work that can be done to improve the project further.

After Chapter 5, References, Appendix and User Manual are provided for completeness.

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction to Lost and Found Systems and Web Applications

A lost and found system is a service that helps people recover misplaced items. In the physical world this is usually a counter or a notice board. In the digital world it becomes a web or mobile application where users can post descriptions of lost or found items and search for matches.

Modern web applications are commonly built as Single Page Applications (SPAs). The frontend runs in the browser and communicates with a backend API. This separation allows independent development, better user experience and easier scaling. For campus-scale systems, a web portal that works on both mobile and desktop browsers is usually preferred over a native mobile app because it does not require installation from an app store.

Real-time features such as chat have become standard expectations. Users no longer want to wait for email replies; they expect instant messaging similar to WhatsApp. Technologies like WebSockets make this possible inside a normal web page.

2.2 Importance of Authentication, Privacy and Real-Time Communication

When students report personal belongings, privacy is critical. Sharing a mobile number on a public group can lead to unwanted calls or messages. Therefore the system must allow communication without revealing identity until both parties agree.

Authentication ensures that only registered campus users can create listings and join chat rooms. JWT (JSON Web Token) is a widely used method for stateless authentication. Passwords must never be stored in plain text; bcrypt provides a secure one-way hashing mechanism.

Real-time communication is important because the finder and the owner often need to exchange location details or verification questions quickly. Socket.IO is a popular library that provides WebSocket-based messaging with automatic fallback and room management.

2.3 Technologies Available

Several technology stacks can be used to build a full-stack web application:

- MERN (MongoDB, Express, React, Node.js) – very popular among students and startups. Good ecosystem and large community.
- Firebase – provides authentication, database and hosting with less backend code, but gives less control over the server logic.
- Django (Python) – strong for rapid development and has excellent admin interface, but real-time chat requires additional packages.
- Spring Boot (Java) – enterprise-grade and very secure, but has a steeper learning curve for a short semester project.
- Next.js full-stack – modern approach, but combines frontend and backend in one framework which can be heavier for beginners.

For this project we selected React + Vite on the frontend and Node.js + Express + Socket.IO on the backend. The reasons are:

- Both of us already had basic familiarity with JavaScript.
- React has a large number of learning resources and component libraries.
- Express is lightweight and perfect for REST APIs.
- Socket.IO integrates easily with Express and provides reliable real-time rooms.
- The same language (JavaScript) can be used on both sides, reducing context switching.
- Deployment on free tiers of Render and Vercel is straightforward.

2.4 Related Work

Several university and open-source lost-and-found systems exist. Most of them provide only listing and search functionality. Very few implement private real-time chat with anonymity. Commercial platforms usually charge a fee or are designed for cities rather than a closed campus community.

Academic projects often stop at CRUD operations on items. Adding authentication, role-based access and secure WebSocket rooms increases the complexity but also makes the project closer to real industry requirements. Our work builds on the basic listing idea and adds the missing privacy and communication layers.

2.5 Summary and Problem Definition

From the literature it is clear that a useful campus lost-and-found portal must provide more than simple posts. It needs secure authentication, structured listings, private communication and an administrative view.

Problem Definition:

To design and implement a full-stack Campus Lost & Found Portal that allows students to report lost and found items, manage their own listings, and communicate through private anonymous chat rooms, while providing an admin overview and following proper authentication and security practices.

2.6 Goals and Objectives

The main goals and objectives of this project are:

1. To understand the requirements of a campus lost-and-found system and the importance of privacy.
2. To learn and apply React, Express, JWT, bcrypt and Socket.IO in a complete application.
3. To implement secure registration, login and role-based access control.
4. To create pages for listing lost items, found items, personal listings and real-time chat.
5. To design anonymous chat rooms so that the identity of participants remains masked.
6. To prepare the code structure (including Mongoose models) for a future MongoDB migration.
7. To deploy the application on free cloud platforms (Render and Vercel).
8. To test the system thoroughly, fix security and routing issues, and document the complete work.
9. To gain practical experience that will be useful in future industry projects.
10. To prepare a proper project report following the college format.

CHAPTER 3

DESIGN FLOW / PROCESS

3.1 Concept Generation

The basic idea of the project is to create a web portal that students can open on their phones or laptops, log in with a university email style account, post a lost or found item, and then chat privately with the other person without revealing phone numbers or real names.

The concept includes:

- Separate pages for Lost items and Found items so that browsing is easy.
- A My Listings page where a student can delete his or her own posts.
- Anonymous chat rooms that are created only when a student claims interest in an item.
- Masked sender names such as Owner #xxx and Finder #xxx so that privacy is preserved.
- An Admin page that shows simple statistics (total users, total items, open chats).
- JWT based authentication so that the backend can trust every request.

The idea was to keep the project achievable within one semester while still demonstrating authentication, real-time communication and security awareness.

3.2 Selection of Technology Stack

Three main stacks were compared before finalising the technology choices.

Table 3.1 Comparison of Available Technology Stacks

Feature	React + Express + Socket.IO	Firebase	Django + Channels
Language	JavaScript (both sides)	JavaScript + proprietary	Python
Real-time chat	Excellent (Socket.IO)	Built-in	Possible with Channels
Learning curve	Moderate	Low	Moderate to high
Free deployment	Render + Vercel	Firebase free tier	Limited free options
Control over backend	Full	Limited	Full
Suitable for semester project	High	High	Medium

React + Express + Socket.IO was selected because it gives full control, uses one language, has excellent free hosting options and matches the skills we wanted to develop.

3.3 Design Constraints

Following constraints were considered while designing the project:

- The complete system must run on free tiers of hosting platforms.
- No paid third-party services (SMS, email gateways, etc.) should be required for the core features.
- The application must work on normal student laptops and mobile browsers.
- Internet connection is required because the chat and API are hosted on the cloud.
- Data persistence is currently in-memory for simplicity; the design must allow easy migration to MongoDB later.
- The report and code must follow the exact format expected by the college.
- Development time is limited to approximately five weeks.

3.4 Alternative Designs

Two alternative approaches were considered before finalising the design:

Alternative 1:
A simple listing website without any chat facility. Students would post items and share their own contact numbers in the description. This approach is easier and faster to build but does not solve the privacy problem and does not demonstrate real-time skills.

Alternative 2:
A complete portal with authentication, lost/found listings, My Listings management, anonymous real-time chat and an admin overview page. This requires more effort but delivers a much more useful and professional system.

3.5 Best Design Selection

Alternative 2 was selected because of the following reasons:

- It solves the real privacy problem by providing anonymous chat.
- It demonstrates important modern skills (JWT, Socket.IO, role-based access).
- It looks more complete and professional during demonstration and viva.
- The extra effort is justified by the learning outcome and the quality of the final product.
- It matches the expectation of a final-year / third-year project in Computer Science.

3.6 Implementation Plan

The implementation was done in the following steps:

1. Create the project structure with separate client (React + Vite) and server (Node + Express) folders.
2. Implement user registration and login APIs with bcrypt hashing and JWT token generation.
3. Create JWT authentication middleware and protect the required routes.
4. Build frontend pages for Login, Register and a protected Dashboard.
5. Implement Item model (in-memory) and CRUD APIs for lost and found listings.
6. Develop My Listings page with delete functionality and correct route ordering.
7. Integrate Socket.IO, create chat rooms on claim, enforce membership checks and generate anonymous labels.
8. Add Admin page with basic statistics and restrict it to users with admin role.
9. Configure environment variables, deploy backend on Render and frontend on Vercel, and connect them.
10. Perform end-to-end testing, fix the discovered issues, take screenshots and write this report.

Figure 3.1 Overall Architecture of Campus Lost & Found Portal

[Architecture diagram showing React frontend ↔ Express API + Socket.IO ↔ In-memory store / future MongoDB]

CHAPTER 4

RESULTS ANALYSIS AND VALIDATION

4.1 Tools Used

The main tools and technologies used in this project are:

- Frontend: React 18, Vite, React Router, Axios, socket.io-client
- Backend: Node.js, Express, Socket.IO, jsonwebtoken, bcryptjs
- Development: Visual Studio Code, Git, GitHub
- Deployment: Render (backend), Vercel (frontend)
- Documentation: Microsoft Word for this report

No paid plugin or extra software was required. Everything was done using free tools and free tiers of cloud platforms. This is important for students who may not have budget for paid services.

4.2 System Architecture and Project Structure

The project is organised into two main folders:

client/

- src/pages – Login, Register, Dashboard, Lost, Found, MyListings, Chat, Admin
- src/components – Navbar, ProtectedRoute, etc.
- src/context – AuthContext for managing login state
- src/api – Axios instance and helper functions

server/

- routes – auth, items, chat, admin
- middleware – auth middleware (JWT verification)
- models – User, Item, ChatRoom, Message (Mongoose schemas prepared for future use)
- store – simple in-memory arrays used for the current demo
- index.js – Express app + Socket.IO server

Figure 4.1 Project Folder Structure

4.3 Modules Implemented

Authentication Module

Students can register with name, email and password. Passwords are hashed with bcrypt before storage. On successful login a JWT is returned and stored on the client. Protected routes on the frontend check for the presence of a valid token. The backend middleware verifies the token on every protected request.

Listings Module

Users can create a lost or found listing with title, description, location and category. All listings can be viewed on separate Lost and Found pages. Only the owner can delete a listing from the My Listings page.

Chat Module

When a student is interested in an item, a private chat room is created between the item owner and the claimant. Sender names are masked (Owner #xxx / Finder #xxx). Socket.IO handles the real-time delivery of messages. A membership check prevents unauthorised users from joining a room.

Admin Module

Users with the admin role can open an overview page that shows simple statistics such as total registered users, total items and number of active chat rooms.

4.4 API Endpoints

Table 4.1 List of API Endpoints

Method	Endpoint	Access	Purpose
POST	/api/auth/register	Public	Register a new student
POST	/api/auth/login	Public	Login and receive JWT
GET	/api/items	Authenticated	Get all items (filter by type)
POST	/api/items	Authenticated	Create a new lost/found item
GET	/api/items/my-listings	Authenticated	Get items created by current user
DELETE	/api/items/:id	Owner only	Delete own item
POST	/api/chat/rooms	Authenticated	Create or get chat room for an item
GET	/api/chat/rooms/:id/messages	Member only	Get previous messages
GET	/api/admin/stats	Admin only	Get overview statistics

4.5 Positive Test Cases

Positive test cases check the normal working of the system when correct input is given.

Table 4.2 List of Positive Test Cases

S.No	Test Case	Expected Result
1	Register with valid details	201 Created, user saved
2	Login with correct credentials	200 OK, JWT returned
3	Create lost item while logged in	201 Created, item appears in list
4	Create found item while logged in	201 Created, item appears in list
5	View all lost items	200 OK, array of items
6	View My Listings	200 OK, only own items
7	Delete own item	200 OK, item removed
8	Open chat room for an item	Room created, messages can be sent
9	Admin views statistics	200 OK, stats object returned
10	Protected page without token	Redirect to login

4.6 Negative and Security Test Cases

Table 4.3 List of Negative and Security Test Cases

S.No	Test Case	Expected Result
1	Login with wrong password	401 Unauthorized
2	Access protected API without token	401 Unauthorized
3	Access admin stats as normal student	403 Forbidden
4	Delete another user's item	403 Forbidden
5	Join a chat room without membership	Connection rejected
6	Register with already used email	400 Bad Request
7	Create item with missing required fields	400 Bad Request
8	Call /my-listings before the :id route (after fix)	Correct list returned

4.7 Environment Variables and Deployment

The following environment variables are used:

- JWT_SECRET – secret key used to sign and verify tokens
- CLIENT_URL – allowed origin for CORS (the Vercel frontend URL)
- VITE_API_URL – base URL of the backend used by the React app

The backend is deployed on Render. The frontend is deployed on Vercel. Both services pull the code from the same GitHub repository. After deployment the live site can be accessed from any browser.

4.8 Issues Found and Fixed

During testing several issues were discovered and fixed. The important ones are listed below.

Table 4.4 Issues Found and Fixed During Testing

Issue	Cause	Fix
Students logged out when opening Dashboard	Dashboard called an admin-only endpoint; 403 was treated like an expired session	Clear session only on 401; call admin endpoint only when user role is admin
My Listings returned "Item not found"	The /:id route was declared before /my-listings	Reordered routes so that /my-listings is matched first
Any logged-in user could join any chat room	No membership check on the join-room event	Added membership verification before allowing a user to join a Socket.IO room
Chat sender labels were wrong for found items	Labels were hardcoded and did not consider item type	Labels are now generated from item type with a stable per-user, per-room number

4.9 Detailed Observations

Some important observations made during the project are listed below:

1. Using an in-memory store is convenient for a demo but data is lost when the server restarts. The Mongoose models are already written so migration to MongoDB will be straightforward.

2. JWT tokens stored in localStorage work well for a student project, but httpOnly cookies would be more secure against XSS in a production system.

3. Route order in Express is critical. Specific paths must be declared before parameterised paths.

4. Socket.IO rooms must always verify that the connecting user is allowed to join; otherwise any authenticated user can listen to private conversations.

5. Clearing the client session only on 401 (and not on 403) prevents unnecessary logouts when a normal user hits an admin endpoint.

6. Anonymous labels (Owner #xxx / Finder #xxx) give enough context without revealing real names.

7. Deploying frontend and backend on two different free services requires careful CORS and environment variable configuration.

8. Writing the report in parallel with development helped us keep the documentation consistent with the actual code.

9. The project can be demonstrated easily by opening the live URL and performing a full flow: register → post item → claim → chat.

10. Negative and security testing revealed more real problems than the happy-path tests alone.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Conclusion

In this project, a full-stack Campus Lost & Found Portal was successfully designed and implemented. Students can register, log in, create lost and found listings, manage their own posts and communicate through private anonymous chat rooms. An admin overview page provides basic statistics.

The frontend was built with React and Vite. The backend uses Node.js, Express and Socket.IO. JWT authentication, bcrypt password hashing and role-based access control were implemented. The application is deployed with the backend on Render and the frontend on Vercel.

The project helped us understand how authentication, real-time communication and security considerations are handled in a real web application. It also gave practical experience of deploying a full-stack system and writing systematic documentation.

Overall the objectives set at the beginning of the project were achieved. The work was completed within the planned timeline and documented properly as per college guidelines.

5.2 Deviation from Expected Results

There were a few deviations from the initial plan. The most important ones were related to authentication handling and route ordering. We initially treated every non-200 response as a reason to log the user out; this caused unnecessary logouts when a normal student opened a page that internally called an admin endpoint. After analysis the client was changed to clear the session only on 401 responses.

Another deviation was the discovery that Express route order matters. The /my-listings endpoint was returning "Item not found" until the routes were reordered. These issues were fixed before the final demonstration and did not affect the overall success of the project.

Apart from these adjustments, the system behaved as expected for the scenarios that were tested.

5.3 Future Work

The project can be extended in the following ways in future:

1. Replace the in-memory store with MongoDB using the already prepared Mongoose models so that data survives server restarts.
2. Add image upload for items so that students can attach a photo of the lost or found object.
3. Implement push or email notifications when a new message arrives in a chat room.
4. Add search and filter functionality on the Lost and Found listing pages.
5. Introduce email verification at registration time.
6. Add rate limiting on authentication and message endpoints to reduce abuse.
7. Store JWT in httpOnly cookies instead of localStorage for better XSS protection.
8. Write automated API and end-to-end tests using tools such as Jest and Playwright.

These enhancements will make the portal more robust and closer to a production-ready campus service.

REFERENCES

1. React Documentation – https://react.dev/
2. Vite Documentation – https://vitejs.dev/
3. Express.js Documentation – https://expressjs.com/
4. Socket.IO Documentation – https://socket.io/docs/
5. JSON Web Token (JWT) Introduction – https://jwt.io/introduction
6. MDN Web Docs – https://developer.mozilla.org/
7. Render Documentation – https://render.com/docs
8. Vercel Documentation – https://vercel.com/docs
9. bcrypt.js – https://github.com/dcodeIO/bcrypt.js
10. React Router Documentation – https://reactrouter.com/

APPENDIX

A.1 JWT Authentication Middleware (sample)

The following middleware verifies the JWT token present in the Authorization header and attaches the decoded user information to the request object.

```
// middleware/auth.js const jwt = require('jsonwebtoken');  function auth(req, res, next) {   const header = req.headers.authorization;   if (!header || !header.startsWith('Bearer ')) {     return res.status(401).json({ message: 'No token provided' });   }   const token = header.split(' ')[1];   try {     const decoded = jwt.verify(token, process.env.JWT_SECRET);     req.user = decoded;     next();   } catch (err) {     return res.status(401).json({ message: 'Invalid or expired token' });   } } module.exports = auth;
```

A.2 Socket.IO join-room Membership Check (sample)

Before a client is allowed to join a chat room, the server verifies that the user is a member of that room.

```
socket.on('join-room', ({ roomId, userId }) => {   const room = chatRooms.find(r => r.id === roomId);   if (!room || !room.members.includes(userId)) {
socket.emit('error', { message: 'Not authorised to join this room' });     return;   }
socket.join(roomId); });
```

A.3 apiFetch Helper (sample)

A small helper is used on the frontend so that the JWT is automatically attached and 401 responses clear the session.

```
async function apiFetch(url, options = {}) {   const token = localStorage.getItem('token');   const headers = { ...options.headers };   if (token) headers.Authorization = `Bearer ${token}`;   const res = await fetch(`${import.meta.env.VITE_API_URL}${url}`, {     ...options,     headers   });
if (res.status === 401) {     localStorage.removeItem('token');
window.location.href = '/login';   }   return res; }
```

USER MANUAL

How to Run the Project Locally

1. Clone the repository from GitHub.
2. Open two terminals – one for the server folder and one for the client folder.
3. In the server folder run: npm install && npm start (or node index.js). Make sure the required environment variables (JWT_SECRET, CLIENT_URL) are set.
4. In the client folder run: npm install && npm run dev. Set VITE_API_URL to the local backend address (usually http://localhost:5000).
5. Open the URL shown by Vite (normally http://localhost:5173) in a browser.
6. Register a new account, log in and explore the features.

How to Use the Live Site

1. Open the deployed frontend URL (Vercel link) in any modern browser.
2. Click Register and create an account with a valid email and password.
3. After registration, log in with the same credentials.
4. On the Dashboard you can navigate to Lost or Found pages.
5. Click "Create Listing" to post a new lost or found item.
6. On any listing you can start a private chat with the owner (or finder).
7. Use My Listings to see and delete the items you have posted.
8. If your account has the admin role, the Admin page will show overview statistics.

Notes

- Because the current demo uses an in-memory store, all data is cleared when the backend server restarts.
- For the best experience use the latest version of Chrome, Firefox or Edge.
- If the chat does not connect, check that the CLIENT_URL environment variable on the backend matches the frontend origin.